BRAC University | **Spring 2026: Midterm Examination**
Department of Computer Science and Engineering | **Set: A**
Course Code: CSE321 | **Duration: 1 Hour 30 Minutes**
Course Title: Operating Systems | **Total Marks: 25**

Name: _____

ID: _____ **Section:** _____

Instructions:

- Answer all questions.
- Marks are shown in brackets, and difficulty is shown as a tag (e.g., CA-E, CA-M, CA-D).
- Understanding the question is part of the exam.
- Write the answers for Questions 1(a), 1(b), 1(c), 2(a), 2(c), 2(d), and 3(a) in the designated answer boxes in this question paper.
- Write the answers for Questions 2(b), 3(b), and 3(c) in the answer script.
- For conceptual questions, write only the precise and relevant answer in short. Do not write detailed or descriptive explanations.

1. CO1 Process [8]

Consider the code snippet given below:

```
1   void sunny() {
2       int x = 0;
3       printf("A ");
4
5       if (fork() == 0) {
6           x += 1;
7
8           printf("B ");
9           if (fork() == 0) {
10              x += 2;
11              printf("C ");
12          } else {
13              wait();
14              x -= 1;
15              printf("D ");
16          }
17
18      } else {
19
20          x += 10;
21          printf("E ");
22      }
23      printf("F ");
24
25  }
```

a) Find out the total number of processes created by this function (including the original process that called sunny)? **CA-M [2]**

Answer:

b) **Identify** which of the following outputs are possible from the code given above? **(Select all possible outputs)**. **CA-M [3]**

(i) A E F B D F C F
(ii) A E F B C F D F
(iii) A B E C F F D F
(iv) A B C F F D E F
(v) A B C F D F E F

Answer:

c) Suppose the **line 23** of the code given above has been changed from **printf("F ");** to **printf("%d",x);**. **Identify** all possible values of **x** that could be printed out after the modification. **(Select all possible outputs)**. **CA-D [3]**

(i) 12
(ii) 0
(iii) 10
(iv) 3
(v) 13

Answer:

2. CO1 CPU Scheduling [11]

a) In an alternate implementation of a multi-level feedback queue, the priority of each process is stored in its PCB (Process Control Block), and a single queue is used instead of having multiple logical queues. All other aspects of the MLFQ remain unchanged.

Explain whether there is any difference in terms of scheduling performance if this alternate version is implemented. **CA-D [2]**

Answer:

b) In an OS, **Multilevel Feedback Queue** functions as a scheduler. In the scheduler, the ready queue is divided into **three** levels. At every level, the round-robin algorithm has been employed, and the following constraints must be maintained.

- **Q1**: time quantum = **3**, priority = **0**
- **Q2**: time quantum = **5**, priority = **1**
- **Q3**: time quantum = **8**, priority = **2**

Here, a lower priority value indicates higher priority; upon completion of an I/O operation, the scheduler will promote a process by one level. The information on burst times, arrival times, and I/O times of three processes is given below: **CA-E [5]**

Process	**Burst Time**	**Arrival Time**	**I/O Time**
P1	8	0	5s I/O [after total 5s CPU allocation]
P2	11	1	N/A
P3	8	2	N/A

(i) **Draw** a Gantt chart using the multilevel feedback queue scheduling algorithm, showing the states of the ready queues of different levels. **[4]**

(ii) **Calculate** the average turnaround time from the Gantt chart. **[1]**

c) A system uses a preemptive scheduling algorithm (SRTF). A process is executing, and a new process arrives with a shorter CPU burst. **CA-E [2]**

(i) **Explain** what happens in this situation in terms of average response time and justify whether this behavior improves system performance based on the mentioned parameters. **[1]**

(ii) **Mention** one possible drawback. **[1]**

Answer:

I	
II	

d) In a data center, tasks are scheduled using priority scheduling. Critical system tasks are assigned high priority, while background tasks (e.g., logging, backups) are assigned low priority. Over time, administrators observe that some background tasks are never executed, even though they have been in the system for a long time. **CA-E [2]**

(i) **Explain** why this situation occurs. **[1]**

(ii) **Propose** a mechanism to resolve it. **[1]**

Answer:

I	
II	

3. CO2 Threads & Process Synchronization [6]

a) A server runs on a system with 4 cores. It receives 8 independent tasks (T1, T2, T3,.., T8), each taking equal time. Two designs are considered:

- **Design A:** Uses thread pool (size = 4)
- **Design B:** Creates 8 new threads dynamically (no pool)

Explain which design will perform better in terms of overhead and efficiency. **CA-M [2]**

Answer:

b) A multimedia application uses multiple threads to handle tasks. **CA-E [2]**

- **Thread 1:** Responsible for processing user inputs.
- **Thread 2:** Performs heavy video rendering.

(i) **Identify** which benefit of multithreading is demonstrated in this scenario. **[1]**

(ii) **Explain** if a single-threaded program would perform better or worse in this situation. **[1]**

c) Consider the following implementation of *compare_and_swap*(CAS) and a function *increment* that uses CAS:

```
1 int CAS(int *v, int expected, int new) {
2     int temp = *v;
3     if (*v == expected)
4         *v = new;
5
6     return temp;
7 }
```

```
1 void increment(int *i) {
2     int old = *i;
3     while (CAS(i, old, old + 1) != old) {
4         old = *i;
5     }
6     print(*i);
7 }
```

Suppose the initial value of i is **0**, and **10** threads concurrently call the *increment* function, passing $\&i$ as the argument. **Write** the output of the program and **justify** it. **CA-D [2]**